

AI ENGINEERING COHORT · DELIVERABLE 2 · WEEK 3

Design Opportunities

Evaluation & Observability

Abhijeet Hiwale

July 2026

[research/week-3/design_opportunities.pdf](#)

Why This Component, and Why Before the Baseline

This one's out of order on purpose. Every other week has scanned a component of the system itself; this week scans how the system gets *measured* — deliberately before Deliverable 3's baseline gets built, not after. A baseline you can't measure precisely isn't a baseline, it's a demo. Seven ideas came out of this scan; six are straightforward adopts. The seventh is the one worth spending the most time on, because it's the one that says something uncomfortable about how this whole field evaluates itself.

1. Recall Numbers Are Close to a Vanity Metric

The finding: models that score near-perfectly on LoCoMo — the most widely reported memory benchmark in the field — drop to 40 to 60 percent on MemoryArena, a benchmark that embeds memory evaluation inside actual multi-step agentic tasks instead of testing recall in isolation. That's not a small gap. That's the difference between "the system found the right fact" and "the system actually used the right fact correctly when it mattered."

Why this matters more than any other finding this week: it's tempting, and honestly the standard practice across most of the field right now, to treat retrieval precision and recall as the finish line. They're not. A memory system can ace every recall benchmark published and still fail the moment that memory has to inform a real decision in a multi-step task. I'm treating this as a genuine limitation of Deliverable 3's baseline rather than pretending around it — the baseline will measure recall, precision, latency, and token efficiency, which is the easier and more tractable half of the picture. Task-level evaluation — does the assistant actually behave correctly because of the memory it retrieved — gets deferred to Deliverable 6, once there's a real system to test it against, not a naive baseline.

The practical takeaway: don't let a clean recall number in Deliverable 3 create false confidence. If anything, this is a reason to keep the eventual design honest about this gap rather than declaring victory on a metric that's known to overstate real capability.

2. The Metric Set Is Already Settled — Just Add One

What's converged: recall, precision, p50/p95 latency, and token efficiency are the standard four across every benchmark and survey scanned this week. There's no reason to invent a different metric set for Deliverable 3 — this one's well-tested and reproducible.

What's missing, and specific to this project: none of the mainstream benchmarks score knowledge updates well — LoCoMo explicitly doesn't, and it's the most-cited one. That's exactly the contradiction-

resolution problem Week 1 flagged as central to this whole system. I'm adding a contradiction/staleness rate as a fifth metric, built on a small labeled set of superseded-preference conversations — the same kind of scenario from Deliverable 1's personal failure evidence.

One methodological rule worth locking in now: LLM-as-judge scoring is fine for "is this answer semantically correct," and not fine for "is this memory still current." Week 1 already found that LLMs are bad at exactly that second judgment, even with explicit instructions. Mixing the two would quietly reintroduce a known failure mode into the eval harness itself.

3. Observability Doesn't Need Custom Tooling — It Needs Discipline

The good news: OpenTelemetry's GenAI conventions have become the de facto tracing standard across the field, and every observability backend worth using — Braintrust, MLflow, OpenObserve — treats OTEL spans as the common substrate. This project doesn't need to invent an instrumentation format. Every memory operation — admission, ranking, supersession, decay, deletion — becomes its own typed span nested under the parent agent run, using tooling that already exists.

The detail that actually matters for this project's privacy constraint: memory content belongs in span *events*, not span *attributes*. Attributes are always indexed and exported; events can be filtered or dropped at the collector level. Combined with collector-level redaction or hashing, this gives a real privacy boundary for close to zero extra engineering — sensitive content stays out of the observability backend by default, rather than needing an after-the-fact scrub every time the policy changes.

One operational note for later, not urgent now: full tracing of every memory operation gets expensive at real volume. Tail-based sampling — keep everything flagged as an error, sample a fraction of routine traffic — is the standard production pattern. Not a concern at MVP scale, but worth designing the span schema so this is a config change later, not a rewrite.

Full matrix of all seven ideas, including the "defer full implementation" call on task-level evaluation, is in *idea_evaluation_matrix.md*. Challenge notes go in *challenge_notes.md* after the live review.